Phần 1. Phân tích bài toán: Cơ chế Tokenizer và Tốc độ xử lý của AI

Mô hình ngôn ngữ lớn (LLM) không đọc văn bản theo từng từ nguyên vẹn hay từng ký tự riêng lẻ, mà xử lý thông qua đơn vị **Token** (được chia tách bởi các thuật toán mã hóa như Byte-Pair Encoding – BPE).

1. Tại sao HTML làm bùng nổ số lượng Token?

- **Thẻ đóng/mở dài dòng:** Để định dạng một đoạn văn bản đơn giản, HTML đòi hỏi cấu trúc cặp thẻ đầy đủ kèm các ký tự đặc biệt như <, >, /.
    - Ví dụ in đậm: `<b>Nấm bệnh</b>` hoặc `<strong>Nấm bệnh</strong>`. Tokenizer phải tách các ký tự `<, strong, >, Nấm, bệnh, <, /, strong, >` thành nhiều token riêng biệt.
    - Khi lồng ghép định dạng (danh sách, bảng biểu, liên kết), số lượng thẻ cấu trúc (`<ul>`, `<li>`, `<table>`, `<tr>`, `<td>`) chiếm tỷ trọng áp đảo so với nội dung thực tế.
- **Markdown tối ưu cú pháp:** Markdown sử dụng các ký tự biểu trưng ngắn gọn (**, *, -, #).
    - Cùng một từ in đậm: `**Nấm bệnh**`. Cặp dấu ** thường chỉ tốn 1 đến 2 token thay vì 4 đến 8 token như thẻ HTML.

2. Tác động trực tiếp đến tốc độ và chi phí

- **Độ trễ sinh văn bản (Output Latency / Time to First Token & Time per Output Token):** LLM sinh từ theo cơ chế tự hồi quy (autoregressive) – đoán từng token nối tiếp nhau. Càng nhiều token cấu trúc rườm rà, mô hình càng tốn chu kỳ tính toán, khiến người nông dân phải chờ đợi câu trả lời lâu hơn.
- **Chi phí API:** Chi phí dịch vụ LLM được tính trên tổng input token và output token. Giảm thiểu các thẻ định dạng không cần thiết giúp cắt giảm trực tiếp 30% – 50% chi phí vận hành trên mỗi lượt hội thoại.

Phần 2. So sánh, Đánh giá & Giải pháp tối ưu

1. Bảng so sánh Markdown và HTML

Tiêu chí	Markdown	HTML

Cú pháp định dạng (VD: In đậm)	`**Bón phân**` (ngắn gọn, ít ký tự thừa)	`<strong>Bón phân</strong>` hoặc `<b>Bón phân</b>` (dài dòng)
Số lượng Token tiêu hao	**Rất thấp:** Chỉ tốn token cho ký tự đánh dấu tối giản (*, -, #).	**Rất cao:** Tốn token cho dấu mở/đóng ngoặc nhọn, tên thẻ, thẻ đóng.
Độ dễ đọc thô (Human-readable)	**Cao:** Người nông dân đọc văn bản thô (raw text) vẫn hiểu trực tiếp nhờ ký hiệu trực quan.	**Thấp:** Dễ rối mắt bởi hàng loạt thẻ code chằng chịt nếu chưa qua trình duyệt render.
Tốc độ xử lý của AI (Latency)	**Nhanh:** Số token sinh ra ít hơn, phản hồi hiển thị nhanh hơn.	**Chậm:** AI mất thời gian sinh thêm các thẻ định dạng rườm rà.
Khả năng linh hoạt giao diện (UI)	Đủ dùng cho nhu cầu văn bản thông thường (tiêu đề, in đậm, danh sách, bảng).	Kiểm soát style chi tiết, nhúng CSS/JS, tùy biến layout phức tạp.
Độ ổn định cú pháp của AI	Ít khi lỗi cú pháp do quy tắc đơn giản.	Dễ gặp lỗi quên đóng thẻ (`<div>`, `<table>`) khi bị ngắt token giữa chừng.

2. Giải pháp tối ưu cho hệ thống "GreenTech"

Giải pháp đề xuất: Tiếp cận kiến trúc "Markdown ở lõi AI – Render phía Client"

1. **System Prompt & Output từ LLM:** Cấu hình hệ thống bắt buộc AI chỉ xuất nội dung dạng **Markdown** chuẩn, nghiêm cấm sử dụng các thẻ HTML lồng ghép.
2. **Kênh truyền thông đa nền tảng:**
    a. **Ứng dụng di động / Web của GreenTech:** Sử dụng thư viện phân tích cú pháp nhẹ ở máy khách (như `react-markdown` hoặc `markdown-it`) để chuyển đổi Markdown thành giao diện trực quan, rõ ràng cho nông dân đọc.
    b. **Kênh tin nhắn văn bản thuần (SMS / Zalo / App bản rút gọn):** Giữ nguyên raw Markdown. Người nông dân vẫn đọc hiểu mạch lạc nhờ các dấu gạch đầu dòng - và in đậm ** mà không bị rối mắt bởi thẻ lập trình.

3. Lập luận lý do lựa chọn

- **Tối ưu triệt để chi phí vận hành:** Với tệp khách hàng nông nghiệp có tần suất hỏi đáp cao nhưng ngân sách dịch vụ giới hạn, việc cắt giảm từ 20% đến 40% lượng token định dạng cho mỗi phản hồi giúp hệ thống tiết kiệm ngân sách máy chủ và API đáng kể.
- **Tối ưu trải nghiệm mạng yếu:** Người nông dân thường ở khu vực đồng ruộng có kết nối 3G/4G chập chờn. Markdown làm dung lượng gói dữ liệu (payload) gửi về nhẹ hơn, tốc độ phản hồi từ AI gần như tức thì, giảm tỷ lệ rớt mạng khi đang nhận câu trả lời.
- **Tính "Human-readable" vượt trội:** Nếu hệ thống gặp sự cố render hoặc gửi qua kênh SMS/tin nhắn thông thường, văn bản Markdown vẫn giữ được tính tự nhiên, gần gũi và dễ hiểu đối với bà con nông dân, trong khi mã nguồn HTML thô sẽ gây khó hiểu và giảm độ tin cậy của dịch vụ.